

Architecture Presentation:

Python Code Understanding SLM Agent

Team Mobtrap: Szárics Iulia, Nimigean Valentin, Şoptelea Sebastian, Balog David

1 Architecture Overview: Weeks 5–6

During this initial phase, we have defined the core infrastructure of our agent. Our goal is to move beyond simple text-matching and create a system that understands Python’s hierarchical nature.

1.1 Dataset Selection: The Training Mixture

To ensure our Small Language Model (SLM) excels at code reasoning, we are going to use a three-tier data strategy:

- **High-Signal Code:** We are utilizing *The Stack v2 (Python Subset)* for diverse repository exposure and *CodeSearchNet* to align code logic with natural language explanations.
- **Logic Benchmarking:** We are going to integrate *HumanEval* and *PyCodeBench* during the SFT (Supervised Fine-Tuning) phase to measure logical consistency.
- **Synthetic Reasoning:** We are going to generate synthetic Q&A pairs from complex dependency graphs to teach the model how different files in a repo interact.

1.2 Chunking Strategy: AST-Based Parsing

We have moved away from character-count chunking. Instead, we are implementing **Abstract Syntax Tree (AST) Chunking**:

- **Structural Integrity:** We are going to split code into logical blocks: Classes, Methods, and Functions.
- **Context Preservation:** We will include decorators, function signatures, and inheritance info in every chunk so the model never loses the "context" of a code snippet.

1.3 Vector Database Model Choices

For our tech stack, we have prioritized a balance between reasoning depth and local performance:

- **Primary Model:** We are going to use **Qwen-2.5-Coder (7B/14B)** as our baseline due to its state-of-the-art performance in code reasoning, with **Phi-4-mini** as a lightweight alternative for basic explanations.
- **Vector Storage:** We have chosen **Qdrant**. Its advanced filtering allows us to query specifically by metadata (e.g., "Find only methods within this specific class").

1.4 RAG Strategies to Try

We are going to experiment with two advanced retrieval methods:

1. **Repo Map Retrieval:** Providing the model with a "bird's eye view" of the entire file structure before fetching specific code.
2. **Hybrid Search:** Combining Vector embeddings (for semantic meaning) with BM25 keyword search (for finding specific variable or function names).

2 Project Roadmap: Weeks 7–12

Following our architecture finalization, we are moving into our implementation sprints:

2.1 Sprinting Foundations (Weeks 7–8)

We are going to build the ingestion pipeline.

- **Week 7:** Implementing the AST parser and metadata attachment.
- **Week 8:** Finalizing the "Repo Map" and hybrid retrieval logic in Qdrant.

2.2 Fine-Tuning & Logic (Weeks 9–10)

We will transition from "retrieving" to "understanding."

- **Week 9:** Integrating the SLM and adding tool-calling capabilities (like *grep*) so the agent can "explore" the repo.
- **Week 10:** Performing QLoRA fine-tuning on our curated Python datasets to improve structured output.

2.3 Alignment & Deployment (Weeks 11–12)

The final weeks focus on reliability and user experience.

- **Week 11:** We are going to use DPO (Direct Preference Optimization) to rank 500–1000 outputs, ensuring the model prefers grounded facts over hallucinations.
- **Week 12:** Model quantization to 4-bit for local speed and the launch of our Streamlit-based UI.